Министерство науки и вышего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

« Московский педагогический государственный университет »

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Выполнил студент группы ЗОМ34-ПВШ2616: **Золотухин Андрей Александрович**

Принял: Жданов Сергей Александрович

Дата сдачи: 10.10.26

Москва

2026

СОДЕРЖАНИЕ

1. Ознакомление с примерами из лекции 3
2. Основная задача 8

1. Ознакомление с примерами из лекции

```
// DynamicArray.cpp
// C++ программа иллюстрирует создание простого класса динамического массива,
// который использует операции со ссылкми на память для хранения данных в виде
массива
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей

#include <iostream>

/* Класс DynamicArray управляет массивом, размер которого
   может динамически увеличиваться.
   Класс имеет указатель на массив arr для хранения элементов массива,
   длину size для хранения величины размера массивар
   и переменную capacity для задания динамики изменения длины массива.
*/


class DynamicArray {
    /*
    Класс DynamicArray начинается с объявления частного
    целочисленного указателя arr, который указывает на динамически выделенный
массив.
    */
private:
    int* arr; // ссылка на элементы массива
    int size; // размер для отслеживания текущего количества элементов в
массиве
    int capacity; // емкость для обозначения общего размера выделенной памяти

public:
    // конструктор класса
    /*
      Конструктор инициализирует динамический массив с начальной емкостью 2.
      Эту начальную емкость можно настроить в зависимости от конкретных
требований программы.
    */
    DynamicArray() : size(0),  capacity(2) {
        arr = new int[capacity];
    }
    // деструктор класса
    /*
     Деструктор гарантирует, что выделенная память будет освобождена,
     когда объект выйдет за пределы области видимости.
    */
    ~DynamicArray() {
        delete[] arr;
```

```
    }
    // методы класса
    // добавить элемент в массив
    /*
     Метод push_back используется для добавления элементов в динамический
массив.
     Если текущий размер массива равен его емкости, это означает, что
выделенная
     память заполнена и размер массива необходимо изменить.
     Затем метод создает новый массив с удвоенной ёмкостью,
     копирует существующие элементы в новый массив, удаляет старый массив и
     обновляет указатель на новый массив.
    */
    void push_back(int element) {
        if (size == capacity) {
            int* newArr = new int[capacity + 1];
            for (int i = 0; i < size; i++)
                newArr[i] = arr[i];
            delete[] arr;
            arr = newArr;
            capacity++;
        }
        arr[size++] = element; // записать элемент в массив
    }
    /*
     Метод at — это метод получения, который возвращает элемент по указанному
индексу.
     Он включает проверку границ для предотвращения доступа к элементам за
пределами
     допустимого диапазона массива.
    */
    int at(int index) const {
        if (index < 0 || index >= size)
            throw std::out_of_range("Index out of bounds");// сообщение об
ошибке
        return arr[index]; // возвращает элемент по его индексу в массиве
    }
    /*
     Методы getSize и getCapacity — это простые функции доступа для получения
     текущего размера и ёмкости динамического массива соответственно.
    */
    int getSize() const {
        return size; // вернуть значение размера массива
    }

    int getCapacity() const {
```

```
        return capacity; // вернуть значение ёмкости массива
    }
};
// Основная программа
/*
 Создаём экземпляр DynamicArray и элементы в массив
 добавляются с помощью метода push_back в цикле.
 Демонстрируем динамическую природу массива, который может расти по мере
необходимости.
*/
int main() {
    DynamicArray dynArray; // Объявить и создать массив dynArray начальной
ёмкости 2

    // записываем элементы в массив (я выбрал квадраты чисел от 1 до 10)
    for (int i = 1; i <= 10; i++)
        dynArray.push_back(i * i);

    // выводим информацию о размере и ёмкости созданного массива
   std::cout << "Размер нашего динамического массива: " << dynArray.getSize()
<< std::endl;
   std::cout << "Ёмкость нашего динамического массива: " <<
dynArray.getCapacity() << std::endl;
    // выводим информацию о индексах и значений элементов массива
    for (int i = 0; i < dynArray.getSize(); i++)
        std::cout << "Элемента индекса " << i << ": " << dynArray.at(i) <<
std::endl;
    return 0;
}
```

```
Размер нашего динамического массива: 10
Ёмкость нашего динамического массива: 10
Элемента индекса 0: 1
Элемента индекса 1: 4
Элемента индекса 2: 9
Элемента индекса 3: 16
Элемента индекса 4: 25
Элемента индекса 5: 36
Элемента индекса 6: 49
Элемента индекса 7: 64
Элемента индекса 8: 81
Элемента индекса 9: 100
```

```cpp
// Stack.cpp
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
// Язык: C++. Иллюстрирует работу такой структуры данных как Стек

#include <iostream>
#include <stack>

int main() {
  // значения, помещенные в стек,
  // должны содержать те же данные, которые записаны во время объявления стека
  std::stack<int> stack; // стек целочисленных данных

  // Заменил ручное добавление в цикл и поменял значения стека немного
  for (int i = 21; i <= 25; i++) {
    stack.push(i); // добавить значение в стек

    // Stack top
    std::cout << " Top = " << stack.top() << std::endl;
  }

  int number = 0;
  stack.push(number);
  std::cout << " Top = " << stack.top() << std::endl; // заголовок стека

  std::cout << " Размер стека = " << stack.size() << std::endl;
  std::cout << " Содержимое стека: ";
  while (!stack.empty()) {
    std::cout << stack.top() << " ";

    // Удаляем элемент стека
    stack.pop();
  }
  std::cout << std::endl;
  if (stack.empty()) // стек пуст?
    std::cout << " Stack is empty." << std::endl;
  else std::cout << " Stack is not empty." << std::endl;
}
```

```
 Top = 21
 Top = 22
 Top = 23
 Top = 24
 Top = 25
 Top = 0
```

```
Размер стека = 6
Содержимое стека: 0 25 24 23 22 21
Stack is empty.
```

2. Основная задача

```cpp
/* Автор: студент группы ЗОМ34-ПВШ2616
 * СДАП Лабораторная работа 4
 * Язык: C++
 */

#include <iostream>
#include <vector>
#include <iomanip>

// Функция для вычисления операции
double calculateOperations(double, char, double);

int main() {
  // Вводим размер последовательностей
  int n;
  std::cout << "Введите n (от 3 до 10 (включительно)): ";
  std::cin >> n;

  if (n < 3 || n > 10)
    // В данном контексте из задания требуется определённый размер
последовательностей из некоторого интервала
    throw std::invalid_argument("Ошибка: число не из нужного диапазона.");

  // Инициализирую и задаю размер n для динамических массивов: двух
последовательной и операций
  std::vector<double> x(n), y(n);
  std::vector<char> o(n);

  std::cout << "Введите " << n << " элементов последовательности x: ";
  for (int i = 0; i < n; i++)
    std::cin >> x[i];

  std::cout << "Введите " << n << " элементов последовательности y: ";
  for (int i = 0; i < n; i++)
    std::cin >> y[i];

  std::cout << "Введите " << n << " символов операций (без пробелов, например
+*-/^): ";
  for (int i = 0; i < n; i++)
    std::cin >> o[i];

  std::cout << std::fixed << std::setprecision(3);
  std::cout << std::endl << "Вычисления на каждом из этапов:" << std::endl;

  // Инициализирую переменную под именем sum и задаю ей значение 0
```

```
  double sum = 0.0;
  try {
    for (int i = 0; i < n; i++) {
      double result = calculateOperations(x[i], o[i], y[i]);
      std::cout << x[i] << " " << o[i] << " " << y[i] << " = " << result <<
std::endl;

      // Прибавляем каждой результат выполнения операции к общей сумме
      sum += result;
    }
  } catch (const std::exception &e) {
    std::cout << "Ошибка: " << e.what() << std::endl;
    return 1;
  }

  // Вывод суммы после всех вычислений
  std::cout << std::endl << "Сумма = " << sum << std::endl;

  return 0;
}

double calculateOperations(double x, char operation, double y) {
  switch (operation) {
    case '+':
      return x + y;
    case '*':
      return x * y;
    case '-':
      return x - y;
    case '/': {
      if (y == 0.0)
        throw std::runtime_error("Ошибка: деление на ноль!");
      return x / y;
    }
    case '^': {
      int result = 1;
      for (int i = 0; i < y; i++)
        result *= x;
      return result;
    }
    default:
      throw std::runtime_error(std::string("Неизвестная операция: ") +
operation);
  }
}
```

```
Введите n (от 3 до 10 (включительно)): 10
Введите 10 элементов последовательности x: 10 9 8 7 6 5 4 3 2 1
Введите 10 элементов последовательности y: 1 2 3 4 5 6 7 8 9 10
Введите 10 символов операций (без пробелов, например +*-/^): */-^+*/-^+

Вычисления на каждом из этапов:
10.000 * 1.000 = 10.000
9.000 / 2.000 = 4.500
8.000 - 3.000 = 5.000
7.000 ^ 4.000 = 2401.000
6.000 + 5.000 = 11.000
5.000 * 6.000 = 30.000
4.000 / 7.000 = 0.571
3.000 - 8.000 = -5.000
2.000 ^ 9.000 = 512.000
1.000 + 10.000 = 11.000

Сумма = 2980.071
```